Canon X-07 – Référence technique

Synthèse des 2e et 3e parties
« Mystères du X-07 » et « Applications »
du livre *Les mystères du Canon X-07*

Ce document reprend les pages 65 à 135 du livre. La première moitié (chapitres 7 à 10) rassemble sous forme de tableaux les informations techniques : architecture, carte mémoire, ports d'entrée/sortie, sous-processeur T6834, codage du BASIC, zone système, routines de la ROM et périphériques. La seconde reprend les huit programmes en langage machine de la 3e partie « Applications », avec leurs listings vérifiés par désassemblage. Les numéros de page (p. xx) renvoient au livre. Sauf mention contraire, les valeurs sont en hexadécimal. Les coquilles et incohérences relevées dans le livre sont signalées par **N.d.R.** . Les notes « vérifié dans la ROM » s'appuient sur le désassemblage du vidage de la ROM BASIC (`ROM_x07/x07.bin`, implanté en `B000`–`FFFF`).

Table des matières

2e partie du livre : Mystères du X-07 **3**

1 Architecture générale (p. 65-67) **3**
1.1 Composants (fig. 18) . . . 3

2 Carte mémoire (p. 67-69) **3**
2.1 Détection des cartes ROM (p. 67-69) . . . 3

3 Ports d'entrée/sortie du HD61L202F (p. 69-74) **4**
3.1 Ports en écriture (fig. 21) . . . 4
3.2 Ports en lecture (fig. 23) . . . 4

4 Le sous-processeur T6834 (p. 74-86) **5**
4.1 Liste des commandes (§ 7.6.1 et fig. 24) . . . 5
4.2 Matrice du clavier (fig. 25) . . . 7
4.3 RAM du T6834 (§ 7.6.2) . . . 7
4.4 Utilisation du T6834 en langage machine (§ 7.6.3) . . . 7

5 Le BASIC en mémoire (p. 87-90) **8**
5.1 Table des mots-clés et points d'entrée en ROM (fig. 27 et 28) . . . 9

6 Zone de travail du système (p. 90-95) **10**
6.1 Vecteurs, RST et hooks . . . 10
6.2 Variables du système . . . 11

7 Principales routines de la ROM (p. 96-110) **12**
7.1 Entrées/sorties . . . 12
7.2 Conversions . . . 14
7.3 Arithmétique . . . 14

7.4 Fonctions mathématiques . . . 15
7.5 Routines système . . . 15
7.6 Routines secondaires . . . 16

8 Périphériques (p. 111-115) **17**
8.1 Cartes mémoire . . . 17
8.2 Imprimante graphique X-710 . . . 17
8.3 Interface vidéo X-720 . . . 18

3e partie du livre : Applications 19

9 Vue d'ensemble 19
9.1 Les huit programmes . . . 19
9.2 Le chargeur BASIC type . . . 19
9.3 Routines de la ROM utilisées . . . 19

10 Affichage d'un motif 8×8 (p. 119-120) **20**

11 Inversion de l'écran vidéo (p. 121-122) **21**

12 Inversion de l'écran LCD (p. 123-124) **21**

13 Affichage d'un titre (p. 125-126) **22**

14 Copyright personnel (p. 127-128) **23**

15 Écriture sur la table traçante X-710 (p. 129-130) **24**

16 Bruitages (p. 131-132) **25**

17 Redéfinition d'une touche (p. 133-134) **26**

18 Conclusion du livre (p. 135) **27**

2e partie du livre : Mystères du X-07 (p. 65-116)

1 Architecture générale (p. 65-67)

Le X-07 est entièrement en technologie CMOS et fonctionne sur 4 piles de 1,5 V. Il utilise deux processeurs :

— un **NSC800** (compatible Z80), le processeur principal ;
— un sous-processeur **Toshiba T6834**, qui gère le clavier, l'affichage, les caractères graphiques, l'horloge, l'alarme, le buzzer, l'alimentation et le contrôle des piles.

Le NSC800 envoie ses commandes au T6834 par le circuit d'E/S **HD61L202F**. Une fois que le T6834 a accusé réception, le processeur principal continue. La mémoire de l'écran LCD n'est pas accessible directement par le Z80 : elle est adressée par le T6834.

1.1 Composants (fig. 18)

Fonction	Composant
Processeur principal	NSC800
RAM interne 8 Ko	4 × HM6117FLP
Support d'extension RAM	8 Ko
Connecteur de carte (« utility card »)	jusqu'à 16 Ko
ROM masquée	16 Ko + 4 Ko (BASIC, 20 Ko au total)
Circuit d'E/S	HD61L202F
Sous-processeur	T6834, avec 2 Ko de RAM (HM6117FLP)
Écran	LCD 20 colonnes × 4 lignes
Pilotes d'affichage	2 × T6781
Bus d'extension	vers l'interface vidéo X-720
Ports	parallèle (Centronics), série, cassette, buzzer, coupleur optique

Extensions possibles : 16 Ko de RAM (support ou carte), carte ROM Canon de 8 Ko, ROM de 4 Ko et RAM vidéo de l'interface X-720.

2 Carte mémoire (p. 67-69)

L'espace d'adressage est de 64 Ko. Au démarrage, une ROM d'amorçage occupe l'adresse `0000`. Elle est ensuite masquée par la RAM interne. L'adressage du support RAM et de la carte s'inverse lorsqu'une RAM additionnelle de 8 Ko est présente, pour que l'espace reste continu quand la carte est retirée.

Adresses	Taille	Contenu
`0000`–`1FFF`	8 Ko	RAM interne (ROM d'amorçage superposée au démarrage)
`2000`–`3FFF`	8 Ko	RAM de la carte (si une ROM est connectée) *ou* support RAM (si une RAM est connectée)
`4000`–`5FFF`	8 Ko	support ROM *ou* RAM de la carte
`6000`–`7FFF`	8 Ko	ROM de la carte
`8000`–`97FF`	6 Ko	RAM vidéo (X-720)
`A000`–`AFFF`	4 Ko	ROM TV (X-720)
`B000`–`FFFF`	20 Ko	ROM BASIC

2.1 Détection des cartes ROM (p. 67-69)

La carte est détectée par logiciel. À l'initialisation, la ROM cherche le mot-clé `love` entre `2000` et `B000`, par pas de `800` (2048 octets). Les 6 octets qui suivent le mot-clé forment 3 adresses de sous-programmes :

— les deux premières sont toujours exécutées ;
— la troisième est exécutée après un `SLEEP`.

3 Ports d'entrée/sortie du HD61L202F (p. 69-74)

Le circuit d'E/S gère le port Centronics, le port série (UART), le buzzer, la cassette, le générateur de bauds, le coupleur optique et le dialogue avec le T6834. Il occupe les ports `F0` à `F7`.

3.1 Ports en écriture (fig. 21)

Port	Rôle	D7	D6	D5	D4	D3	D2	D1	D0
`F0`	contrôle des interruptions	OFFRQ	SETPS	BIE	ALMIE	XTIE	XRIE	TIE	RIE
`F1`	XBTR : donnée vers le T6834	octet							
`F2`	bauds, poids faible	bits 0–7 du compteur							
`F3`	bauds, poids fort	–	–	–	–	bits 8–11			
`F4`	modes	SETBC	BRGST	CNTR	LEO	MD1	MD0	BZON	REM
`F5`	RAZ des interruptions	RALRM	CSTB	CSFT	RBGI	RTDRE	RRDRF	RXTRE	RXRRF
`F6`	configuration UART	–	IR	RTS	ER	SBRK	RXE	–	TXEN
`F6'`	format série (2e écriture)	S2	–	EP	PEN	–	L1	B2	FX04
`F7`	donnée série à émettre	octet							

Port `F0`. OFFRQ : coupure de l'alimentation ; SETPS : mode économie (éteint) ; BIE : interruption du générateur de bauds ; ALMIE : interruption d'alarme ; XTIE / XRIE : interruptions du bus vers le T6834 ; TIE / RIE : interruptions d'émission / de réception série. Attention : modifier `F0` sans précaution peut bloquer la machine.

Ports `F2`/`F3`. Compteur de 12 bits du générateur de bauds : les 8 bits de poids faible dans `F2`, les 4 bits de poids fort dans les bits bas de `F3`.

Port `F4`. SETBC = 1 charge `F2`/`F3` dans le générateur ; BRGST = 1 lance le comptage, 0 l'arrête ; CNTR : bit envoyé vers la prise parallèle ; LEO = 1 envoie 38,4 kHz au coupleur optique ; BZON : buzzer ; REM = 1 active la télécommande du magnétophone.

MD1	MD0	Mode (fig. 22)
0	0	RAZ du compteur de bauds, arrêt
0	1	liaison série
1	0	cassette (signal sur la prise K7)
1	1	son sur le buzzer

Port `F6` (écriture). IR : initialisation de l'UART ; RTS = 1 met la broche RTS du HD61L202F à 0 ; ER : remise à zéro des indicateurs d'erreur PE, OE et FE ; SBRK : force la ligne à 0 (break) ; RXE : autorise la réception ; TXEN : autorise l'émission.

3.2 Ports en lecture (fig. 23)

Port	Rôle	D7	D6	D5	D4	D3	D2	D1	D0
`F0`	interruptions	TRQ1	TRQ0	BIE	ALMIE	XTIE	XRIE	TIE	RIE
`F1`	XBRR : donnée du T6834	octet							
`F2`	indicateurs	CBSY	ALRM	AUX1	AUX2	0	BGI	XTRE	XRRF
`F4`	modes	FCAS	BRGST	CNTR	LEO	MD1	MD0	BZON	REM
`F6`	état de l'UART	CTS	–	FE	OE	PE	TXE	RXRDY	TXRDY
`F7`	donnée série reçue	octet							

Les ports `F3` et `F5` ne sont pas utilisés en lecture.

— `F0` : TRQ0 = 1 si la touche BREAK est enfoncée ; TRQ1 = 1 est positionné par le T6834 : le NSC800 ne peut alors pas passer en mode sauvegarde.

— `F2` : CBSY : port parallèle occupé ; ALRM = 1 : piles épuisées (testé toutes les 4 minutes par le sous-processeur) ; AUX2 : pile de la carte mémoire ; AUX1 = 0 : interrupteur de la prise d'extension sur RAM ; BGI : sortie du générateur de bauds ; XTRE = 1 quand le T6834 a lu XBTR ; XRRF = 1 quand le T6834 a écrit XBRR.
— `F4` : FCAS : reflet du signal lu sur la cassette.
— `F6` : la donnée série est émise si CTS = 0 et TXEN = 1 ; FE : bit de stop manquant ; OE : donnée reçue avant la lecture de la précédente (overrun) ; PE : erreur de parité ; TXE : registre d'émission vide.

N.d.R. Le livre donne « T*RDY : réception prête, R*RDY : transmission prête » ; les libellés semblent inversés (RXRDY = réception prête, TXRDY = émission prête).

4 Le sous-processeur T6834 (p. 74-86)

Le T6834 accepte 70 commandes (codes `01` à `46`), envoyées par le NSC800 à travers le HD61L202F. Une commande peut être suivie de paramètres. Si nécessaire, le T6834 répond par une suite d'octets.

4.1 Liste des commandes (§ 7.6.1 et fig. 24)

« Env. » est le nombre d'octets envoyés, code de commande compris ; « Rép. » est le nombre d'octets de la réponse ; n désigne une chaîne de longueur variable, terminée par `00`.

Code	Nom	Env.	Rép.	Description
`01`	TIME CALL	1	8	lit l'heure : année (2 octets), mois, date, jour de la semaine, heure, minutes, secondes
`02`	STICK	1	1	touche curseur : `30` aucune, `31` haut, `32` droite, `36` bas, `37` gauche
`03`	STRIG	1	1	0 si F6 est enfoncée, sinon `FF`
`04`	STRIG1	1	1	0 si la barre d'espace est enfoncée, sinon `FF`
`05`	RAM READ	3	1	lit la RAM du T6834 (adresse : octet haut, octet bas)
`06`	RAM WRITE	4	0	écrit dans la RAM du T6834 (adresse haute, basse, donnée)
`07`	SCROLL SET	3	0	première et dernière ligne de défilement
`08`	SCROLL EXEC	1	0	fait défiler l'écran (le curseur ne bouge pas)
`09`	LINE CLEAR	2	0	efface la ligne indiquée
`0A`	TIME SET	9	0	règle l'horloge : seconde, minute, heure, jour, date, mois, année (2 octets)
`0B`	calcul du jour	1	0	calcule et règle le jour de la semaine d'après TIME SET
`0C`	ALARM SET	9	0	règle l'alarme, même format que `0A` (le 2e paramètre est ignoré)
`0D`	BUZZER OFF	1	0	pas de buzzer à l'heure d'alarme
`0E`	BUZZER ON	1	0	buzzer à l'heure d'alarme
`0F`	BIT IMAGE	2	120	renvoie une ligne de l'écran (0 à 3) au format « bit image »
`10`	POINT	3	1	0 si le point (X, Y) est allumé, sinon `FF`
`11`	PSET	3	0	allume le point (X, Y)
`12`	PRESET	3	0	éteint le point (X, Y)
`13`	PEOR	3	0	inverse le point (X, Y)
`14`	LINE	5	0	trace un segment ; paramètres X_1, Y_1, X_2, Y_2
`15`	CIRCLE	4	0	trace un cercle ; paramètres X, Y, r
`16`	UDK WRITE	2+n	0	définit une touche de fonction (numéro 1 à 12, chaîne : 3 caractères de repérage + 38 caractères ; 42 caractères pour F6 et F12)
`17`	UDK READ	2	n	lit la définition d'une touche de fonction
`18`	UDK ON	1	0	le T6834 commence à envoyer les UDK
`19`	UDK OFF	1	0	le T6834 cesse d'envoyer les UDK
`1A`	UDC WRITE	10	0	définit un caractère (codes `80`–`9F` et `E0`–`FF`) : code + 8 octets (6×8 points)
`1B`	UDC READ	2	8	lit un motif de caractère (codes `20`–`FF`)
`1C`	UDC INIT	1	0	rétablit les caractères par défaut

Code	Nom	Env.	Rép.	Description
1D	SP WRITE	1+n	0	écrit le programme de lancement (511 octets au plus, terminé par 00)
1E	SP WRITE CONT	1+n	0	ajoute à la suite de 1D
1F	SP ON	1	0	autorise le programme de lancement
20	SP OFF	1	0	invalide le programme de lancement
21	SP READ	1	n	lit le programme de lancement
22	ON STATE	1	1	bit 0 = 1 : OFF/ON ; bit 6 = 1 : SLEEP ON
23	OFF REQUEST	1	0	coupe l'alimentation
24	LOCATE	4	0	place le curseur en (X, Y) et affiche le caractère (00 : aucun caractère)
25	CURSOR ON	1	0	affiche le curseur
26	CURSOR OFF	1	0	cache le curseur
27	TEST KEY	3	1	teste une ligne de la matrice du clavier (2 octets, voir fig. 25)
28	TEST CHR	2	1	0 si la touche du caractère ASCII est enfoncée, sinon FF (sans SHIFT)
29	RAZ secondes	1	0	remet les secondes à zéro
2A	DATE SET	2	0	écrit la date
2B	DISPLAY OFF	1	0	coupe le signal commun du LCD (la mémoire d'affichage reste accessible)
2C	DISPLAY ON	1	0	rétablit l'affichage
2D	KEY BUFFER CLEAR	1	0	vide le tampon du clavier (sans toucher aux UDK ni au SP)
2E	CLS	1	0	efface l'écran
2F	HOME	1	0	curseur en $(0, 0)$
30	UDK DISP ON	1	0	affiche les étiquettes des UDK sur la ligne 3
31	UDK DISP OFF	1	0	efface les étiquettes des UDK
32	REPEAT KEY ON	1	0	active la répétition des touches
33	REPEAT KEY OFF	1	0	désactive la répétition
34	KANA SET	1	0	assigne les caractères KANA aux UDK
35	UDK CONT WRITE	2+n	0	ajoute des caractères à la définition d'une touche de fonction
36	ALARM READ	1	8	lit l'alarme (même format que 01)
37	BUZZER ZERO	1	1	0 si le compteur est à 0, sinon FF
38	CLICK OFF	1	0	désactive le « clic » des touches
39	CLICK ON	1	0	active le « clic » des touches
3A	LOCATE CLOSE	1	0	ramène le curseur dans la zone de défilement (après 24)
3B	KEY OUT ON	1	0	autorise l'entrée des touches vers l'unité centrale
3C	KEY OUT OFF	1	0	bloque l'entrée des touches vers l'unité centrale
3D	SP AUTO ON	1	0	exécute le programme de lancement à la mise sous tension
3E	SP AUTO OFF	1	0	n'exécute plus le programme de lancement à la mise sous tension
3F	REQUEST SLEEP	1	0	coupe l'alimentation en mode SLEEP
40	UDK INIT	1	0	rétablit les touches de fonction par défaut
41	CHAR WRITE	9	0	affiche un motif 6×8 au curseur (8 octets, format 1A) ; le curseur avance
42	CHAR READ	1	8	renvoie les 8 octets du motif sous le curseur
43	SCANR	3	2	points de la zone 6×8 au curseur (format 1B)
44	SCANL	3	2	nombre de points allumés à gauche de $(X - 1, Y)$
45	TIME CHK	1	1	vérifie l'horloge : bit 0 secondes, 1 minutes, 2 heures, 3 jour, 4 mois, 5 année (1901–2099) ; un bit à 1 signale une erreur ; bits 6 et 7 à 0
46	ALM CHK	1	1	vérifie l'alarme

N.d.R. Le livre écrit « 38 CLICK OFF déclenche » et « 39 CLICK ON enclenche » ; on retient OFF = désactivé, ON = activé.

N.d.R. Vérifié dans la ROM : STICK (D7E3) envoie 82 (commande 02, bit 7 = aucun paramètre) et garde la réponse AND $0F. STRIG(0) (D7F8) envoie 84 (barre d'espace) et STRIG(1) 83 (F6) ; la réponse est inversée par CPL : −1 si la touche est enfoncée, 0 sinon. Les deux fonctions vident ensuite les tampons (C0BD).

4.2 Matrice du clavier (fig. 25)

Pour la commande 27 (TEST KEY), on envoie la valeur de la ligne d'échantillonnage sur 2 octets ; le T6834 renvoie l'état des colonnes K1 à K8.

Ligne	Valeur	K1	K2	K3	K4	K5	K6	K7	K8
A1	0200	SFT	CTR						
BZ	0100	Home		GRP	Num	OFF			ON
S8	0080	L, +, *, },), |, =, – (ponctuation)							
S7	0040	>, ?/×, Yen, CR, O, P, @, { (ponctuation)							
S6	0020	F1	F2	F3	F4	F5	F6		
S5	0010	1	2	3	4	5	6	7	8
S4	0008	Q	W	E	R	T	Y	U	I
S3	0004	A	S	D	F	G	H	J	K
S2	0002	Z	X	C	V	B	N	M	
S1	0001	INS	DEL	→	←	↑	↓	SPC	

Les lignes S7 et S8 sont difficiles à lire sur le scan : on n'en donne que les touches principales.

4.3 RAM du T6834 (§ 7.6.2)

Le T6834 dispose de 2 Ko de RAM, qu'on lit et écrit par les commandes 05/06 ou par les routines E348/E334.

Adresse	Contenu
C006	bit 0 : ONSTATE ; bit 6 : état SLEEP ; bit 7 : mode ALARM simulé
C00C	mode du clavier : 0 normal, 1 KANA, 2 graphique, 3 numérique
C01A	bit 0 = 1 : alarme validée
C040–C04E	heure courante, une valeur tous les 2 octets : secondes, minutes, heures, jour de la semaine, date, mois, année
C050–C05E	alarme, même format
C080	indicateur du curseur
C082	indicateur des touches de fonction utilisateur
C084	compteur du buzzer
C092	compteur de la minuterie d'alimentation
C096	fin de la zone de défilement
C09E	début de la zone de défilement
C800–C9FF	touches de fonction (512 octets, voir ci-dessous)
CA00–CBFF	caractères définis : codes 80–9F en CA00 + $(c - 80) \times 8$; codes E0–FF en CB00 + $(c - E0) \times 8$
CC00–CCFF	tampon du clavier (256 octets, 127 touches)
CE00–CFFF	programme de lancement (512 octets, START$)
E000–E1FE	mémoire de l'écran LCD (511 octets)

Adresses des touches de fonction (fig. 26).

F1	C800	F2	C82A	F3	C85A	F4	C87E	F5	C8A8	F6	C8D2
F7	C900	F8	C92A	F9	C954	F10	C97E	F11	C9A8	F12	C9D2

N.d.R. Avec un pas de 2A, on attendrait C854 pour F3 (comme F9 = C954) ; la valeur C85A du livre est probablement une coquille.

4.4 Utilisation du T6834 en langage machine (§ 7.6.3)

La ROM fournit deux routines principales :

— E428 : envoie une commande **sans paramètre ni réponse**. Charger le code dans A, puis CALL $E428. Modifie A, F et C (L selon le § 7.6.3).

— C92F : envoie une commande **avec paramètres et réponse**. A = code de la commande ; HL = adresse des paramètres ; B = nombre de paramètres ; C = nombre d'octets de la réponse ; DE = tampon de réponse (en général 026E). Modifie AF, BC, DE et HL. Si $B = 0$, le bit 7 de A doit être positionné.

Exemple : faire clignoter l'écran.

```
        LD   B,$FF
loop:   LD   A,$2B       ; DISPLAY OFF
        CALL $E428
        LD   A,$2C       ; DISPLAY ON
        CALL $E428
        DJNZ loop
        RET
```

N.d.R. Le livre écrit `LD B,$2B` à la 2e ligne ; il faut `LD A,$2B`.

Routine d'envoi d'un octet (livre). Cette routine envoie au T6834 l'octet contenu dans `E`, sans modifier aucun registre. La routine ROM `C9C0` attend que le T6834 soit prêt.

```
        PUSH AF
        PUSH DE
        PUSH BC
        LD   C,$F1       ; port de sortie vers le T6834
        CALL $C9C0       ; attente : T6834 prêt ?
        LD   A,($026C)   ; image du port F0 (INTIMAG)
        OR   $80
        OUT  ($F0),A
        OUT  (C),E       ; octet -> port F1
        LD   A,$02
        OUT  ($F5),A
        POP  BC
        POP  DE
        POP  AF
        RET
```

N.d.R. Le listing du livre porte `CALL $C0C9` et `POP HL` ; le code hexadécimal du programme de démonstration (`CD C0 C9`, puis `F1`) montre qu'il s'agit de `CALL $C9C0` et de `POP AF`.

Programme de démonstration (BASIC). Il implante la routine ci-dessus en `1C25`, efface l'écran (`CALL $CE9E`), trace un cercle (commande `15`, $X =$ `3C`, $Y =$ `0F`, $r =$ `0A`), puis fait clignoter l'écran 255 fois (commandes `2B`/`2C`).

```
 5 REM programme BASIC implantant une routine en LANGAGE MACHINE
10 CLS : PRINT "Je charge les codes ... Un instant, SVP !"
20 FOR X = &H1C00 TO &H1C3E
30 READ A$ : POKE X,VAL("&H" + A$)
40 NEXT X
45 REM donnees de la routine LANGAGE MACHINE
50 DATA CD,9E,CE,1E,15,CD,25,1C
60 DATA 1E,3C,CD,25,1C,1E,0F,CD
70 DATA 25,1C,1E,0A,CD,25,1C,06
80 DATA FF,1E,2B,CD,25,1C,1E,2C
90 DATA CD,25,1C,10,F4,F5,C5,D5
92 DATA 0E,F1,CD,C0,C9,3A,6C,02
95 DATA F6,80,D3,F0,ED,59,3E,02
98 DATA D3,F5,D1,C1,F1,C9,00,00
```

Lancement : `EXEC &H1C00`.

5 Le BASIC en mémoire (p. 87-90)

Les programmes BASIC commencent à l'adresse **1363** (décimal), soit `0553`. La zone de 0 à 1362 est la zone système (§ 6). Chaque mot-clé est codé par un octet entre `80` et `FF` (« token »).

Format d'une ligne.

Octets	Contenu
2	adresse de la ligne suivante (poids faible, poids fort)
2	numéro de ligne (poids faible, poids fort)
n	tokens et caractères ASCII
1	`00` : fin de ligne

Exemple du livre. Le programme `1 REM **`, `200 CLS:A$="C7"`, `9999 PRINT"END","C7":END` donne, à partir de l'adresse 1363 : `92 05 01 00 8E 20 2A 2A 00 | 6A 05 C8 00 A6 3A 41 24 DD 22 43 37 22 00 | 7C 05 0F 27 9F ... 80 00` (en hexadécimal ; le livre donne ces valeurs en décimal).

5.1 Table des mots-clés et points d'entrée en ROM (fig. 27 et 28)

L'interpréteur BASIC occupe `B000`–`FFFF`. Le tableau donne le token de chaque mot-clé et l'adresse de la routine correspondante en ROM. Les adresses `0087`–`009C` sont des « hooks » en RAM (zone système), redirigés par l'interface X-720.

Mot-clé	Tok.	Entrée	Mot-clé	Tok.	Entrée	Mot-clé	Tok.	Entrée
END	80	D2B0	FOR	81	F3F7	NEXT	82	D30C
DATA	83	F66A	INPUT	84	F82F	DIM	85	B005
READ	86	F86B	LET	87	F685	GOTO	88	F621
RUN	89	F5B7	IF	8A	F797	RESTORE	8B	D296
GOSUB	8C	F610	RETURN	8D	F63D	REM	8E	F666
STOP	8F	D2C0	ELSE	90	F666	TR	91	D1B7
MOTOR	92	E0B9	DEFSTR	93	F54E	DEFINT	94	F551
DEFSNG	95	F554	DEFDBL	96	F557	LINE	97	F7C6
ERROR	98	F78C	RESUME	99	F734	OUT	9A	E98C
ON	9B	F6F1	LPRINT	9C	E9DE	DEFFN	9D	FC96
POKE	9E	FC80	PRINT	9F	EA05	CONT	A0	D178
LIST	A1	FE75	LLIST	A2	FE6B	CLEAR	A3	D1D0
CIRCLE	A4	009C	CONSOLE	A5	E243	CLS	A6	CE9E
COLOR	A7	008A	EXEC	A8	CEB8	LOCATE	A9	E212
PSET	AA	008D	PRESET	AB	0090	OFF	AC	C5B1
SLEEP	AD	C59B	DIR	AE	E5E3	DELETE	AF	E682
FSET	B0	BF36	PAINT	B1	0099	LOAD	B2	DF38
SAVE	B3	DEFE	INIT	B4	E699	ERASE	B5	E3CF
BEEP	B6	C2CD	CLOAD	B7	DF30	CSAVE	B8	DEF9
NEW	B9	D214	FN	BC	FCB5	ERL	BE	FAC4
ERR	BF	FAB6	STRING$	C0	D835	INSTR	C1	DCD6
INKEY$	C2	C103	INP	C3	E931	VARPTR	C4	E9A7
USR	C5	E9BB	SNS	C6	E961	ALM$	C7	D928
DATE$	C8	D8CE	TIME$	C9	D868	START$	CA	DA99
FONT$	CB	D739	KEY$	CC	DA39	SCREEN	CD	0087
NOT	CF	FC1A	STEP	D0	F537	EQV	D9	FC72
AND	D6	FC68	SGN	DF	C9F1	INT	E0	CC23
ABS	E1	C9DC	FRE	E2	DDD7	POS	E3	FC8C
SQR	E4	B694	RND	E5	B771	LOG	E6	B609
EXP	E7	B6DD	COS	E8	B530	SIN	E9	B549
TAN	EA	B593	ATN	EB	B5AC	PEEK	EC	FC7A
CINT	ED	CAE0	CSNG	EE	CB08	CDBL	EF	CB90
FIX	F0	CC14	LEN	F1	D72D	HEX$	F2	D538
STR$	F3	D53D	VAL	F4	DCB3	ASC	F5	D81A
CHR$	F6	D828	TKEY	F7	D7CC	LEFT$	F8	DC5A
RIGHT$	F9	DC8A	MID$	FA	DC93	CSRLIN	FB	CEAF
STICK	FC	D7E3	STRIG	FD	D7F8	POINT	FE	0093

Codes de contrôle de la fig. 27 : BELL (sonnerie), BS (retour arrière), LF (saut de ligne), LU (ligne précédente), CLS, CR, flèches de déplacement du curseur, DC1/DC2 (passage texte/graphique sur la X-710), SI/SO (caractères de la X-710 en ROM externe/interne).

6 Zone de travail du système (p. 90-95)

La zone système occupe 0000–0552 (1363 octets) :

— 0000–00AE : zone initialisée de façon conventionnelle ;
— 00AE–00B4 : initialisée à la mise sous tension ;
— 00B4–00BE : initialisée à la remise à zéro du BASIC ;
— 00BE–0552 : zone générale de travail du système.

6.1 Vecteurs, RST et hooks

Adresse	Instr.	Rôle
000	C9	RST 00 (point d'arrêt de la carte moniteur XP140F)
003		nombre de caractères imprimés × taille
004	50	longueur de ligne de la X-710 (80)
005		taille des caractères de la X-710
008	JP F52F	RST 08 : comparaison de symbole
00C-00D		début de la ROM 1
010	JP F537	RST 10 : test de caractère
013-014		fin de la ROM 1
018	JP C9C7	RST 18 : émission d'une commande vers le T6834
020	JP EE45	RST 20 : comparaison de HL et DE
024-025		début de la ROM 2
026-027		fin de la ROM 2
028	JP E88F	RST 28 : émission de A vers le périphérique ouvert
02B		BREAKFLAG
02C	C9	INTERRUPT C (clignotement du curseur)
030	JP FC2F	RST 30 : test du type de donnée
034	JP C7A3	INTERRUPT B (interruptions série)
038	JP E906	RST 38 : calcul d'adresse dans un tableau
03C	JP C799	INTERRUPT A (interruptions du T6834)
03F	JP CED6	appel de GPRPUT (imprimante parallèle)
042	JP E512	émission de A vers KBD, CASI
045-046		pointeur de la table du RST 38
047-056		table des hooks
057	JP E327	minuterie automatique
05A	JP C16C	retour
05D	JP C1FA	
060	JP C181	
063	JP C219	
066	C9	NMI
069	JP C582	
06C	JP C0A1	affichage ou masquage du curseur
06F	JP C210	
072	JP C231	affiche C à la position du curseur
075	JP C250	SCREEN / image miroir
078	JP C154	
07B	JP C138	
07E	JP C120	
081	JP C24B	TXTTAB
084	JP F1AA	SN ERROR
087	JP E403	SCREEN
08A	JP F1AA	COLOR (X-720 Péritel), sinon SN ERROR
08D	JP CDF4	PSET
090	JP CDF7	PRESET
093	JP CE05	POINT
096	JP CE19	LINE
099	JP F1AA	PAINT (X-720 Péritel), sinon SN ERROR
09C	JP CE32	CIRCLE
09F	JP C1BE	sortie CONSOLE (LCD par défaut)
0A2	JP C90A	entrée CONSOLE (clavier par défaut)
0A5	JP C16D	
0A8	C9	hook ABORT (JP 7535 avec la carte XP140F)

Adresse	Instr.	Rôle
0AB	C9	hook ERREUR (JP 756C avec la carte XP140F)

La zone 057–09E relie l'afficheur LCD et l'afficheur vidéo Péritel (X-720).

N.d.R. Pour le RST 18, la table indique JP C9C7, alors que le § 9.5.1 donne C937. Vérifié dans la ROM : au démarrage à froid, la routine C3D8 copie AE octets de C6DB vers 0000 (LDIR) ; on y lit C3 C7 C9 à la position 18. **RST 18 saute donc en C9C7**, et C937 tombe au milieu d'une instruction. Le rôle indiqué est en revanche faux : C9C7 ne s'adresse pas au T6834, elle donne le **signe de l'accumulateur flottant** (044E) : A = 0 et Z = 1 si la valeur est nulle, A = 1 si elle est positive, A = FF si elle est négative.

6.2 Variables du système

Adresse	Contenu
0B1	indicateur RESUME en cours
0B2-0B3	TXTTAB : début du programme BASIC
0B8 / 0B9	position du curseur : X / Y
0BA	longueur d'une ligne de l'écran LCD
0BB / 0BC	première / dernière ligne de défilement
0BD	nombre de lignes de défilement
0D5-1D4	tampon d'entrée
1D8	indicateur de variable (0 : elle existe ; FF : à créer)
1D9	type de donnée : 02 entier (%), 03 chaîne ($), 04 simple précision (!), 08 double précision (#)
1DB-1DC	dernière ligne exécutée
1DD-1DE	STKTOP : fin de la pile
1DF-1E0	MEMSIZ : fin de la zone BASIC
1E1-1E2	prochaine entrée libre de la table des chaînes
1E3-200	table de manipulation des chaînes
201	longueur de la chaîne (0 à 255)
202-203	adresse de la chaîne
204-205	FRETOP : prochaine adresse libre de la zone des chaînes
206-207	pointeur du dernier code exécuté / taille des tableaux / indicateur de PRINT USING
208-209	D43A
20A-20B	pile des boucles FOR-NEXT
20E	indicateur de l'instruction FOR
20F	indicateur utilisé par C977
210-211	RAMSTR : début de la zone des fichiers en RAM
212-213	RAMEND : fin de la zone des fichiers en RAM
214-263	image miroir de l'écran LCD (80 octets = 20 × 4)
264-265	sauvegarde de HL (pointeur BASIC) lors d'un SLEEP
266-267	sauvegarde de SP lors d'un SLEEP
268-26B	mot-clé (jour, heure, minute, seconde) placé en RAMSTR−4 et RAMEND−1
26C	INTIMAG : image du port F0
26D	nombre d'octets de réponse attendus du T6834
26E-	1er tampon de communication avec le T6834
2C3-2C4	adresse du pilote de sortie
2C5-2E8	KBNTAB : table de INIT# (5 × 8 octets : adresse + sauvegarde de HL, DE, BC)
2F5	numéro du périphérique ouvert
2F6-2F7	adresse du pilote
2F8-2FD	indicateurs de ALM$, TIME$ et DATE$ (DATE$: 0 si actif, 1 si ALM$ actif)
2FE	indicateur de CLOAD / CLOAD ?
2FF-304	nom du fichier cassette cherché (6 octets)
305-30A	nom du fichier lu sur la bande (SKIP ou FOUND)
30B	UCRIMAG
30C	RSERFLAG
30D	1 si un INPUT est en cours, 0 sinon
30E	indicateur de l'instruction READ
30F-310	appel à l'adresse 0B2 par RUN
311-312	numéro de la ligne en cours d'exécution
313-314	sauvegarde de SP pendant que l'interpréteur travaille
317-318	numéro de la ligne où l'erreur s'est produite

Adresse	Contenu
319-31A	adresse du traitement d'erreur (ON ERROR)
31B	FF si un ON ERROR est actif, 0 sinon
31C-31D	adresse du point décimal dans le tampon de PRINT
31E-31F	adresse de reprise pour CONT
320-321	adresse du dernier code exécuté lors d'une erreur
322-323	VARTAB : table des variables
324-325	ARRTAB : table des tableaux
326-327	STREND : début de la zone libre
328-329	pointeur des DATA après un RESTORE n
32A-343	types des variables par défaut (26 lettres)
344-345	PRMSTK
348-3AB	pile (appel par B07C)
3AC-3AD	PRMPRV
414	indicateur de recherche de variable
415-416	adresse de la variable recherchée
41E-	tampon interne de PRINT
44E-455	accumulateur flottant (44E : exposant ; 450-451 : entier)
456	octet de travail de l'accumulateur
49F-4A6	accumulateur 2
4A7	octet de travail de l'accumulateur 2
4AF-	tampon de RND
4C6-4C9	anciennes coordonnées X, Y de LINE
4CA-4CD	nouvelles coordonnées X, Y de LINE ( )–( )
507-	2e tampon de réponse du T6834
54D	indicateur de présence de la carte XP140F / fonction LOG
54E-54F	début de la zone de travail de la carte moniteur XP140F

N.d.R. Vérifié dans la ROM : `0B8` **contient la ligne et** `0B9` **la colonne**, toutes deux comptées **à partir de 1**. `LOCATE x,y` (`E212`) charge H = x+1 et L = y+1 puis exécute `LD ($00B8),HL` (`C164`) ; `CSRLIN` renvoie (`0B8`)−1 et `POS` (`0B9`)−1 ; un retour chariot remet `0B9` à 1. Les programmes du livre qui écrivent `LD HL,$0502 : LD ($00B8),HL` placent donc le curseur en colonne 5, ligne 2.

N.d.R. Vérifié dans la ROM : à chaque `RUN` (`D234`), la ROM remplit les 26 octets `032A`–`0343` avec `LD (HL),$08` : **une variable sans suffixe est en double précision** (14 chiffres), comme sur les MSX, et non en simple précision. `PRINT 1/3` affiche donc `.33333333333333`.

BREAKFLAG (`002B`). Bit 0 : BREAK ; bit 1 : piles épuisées ; bit 3 : pile de la carte épuisée ; bit 6 : arrêt par la minuterie ; bit 7 : OFF.

N.d.R. Vérifié dans la ROM : à chaque instruction, l'interpréteur appelle `C000`, qui traite le bit 7 (extinction, par `C5A6`), le **bit 2 (pause** jusqu'à l'appui sur une touche, non cité par le livre), les bits 1 et 3 (messages de piles faibles, par `C06C`) puis le bit 0 (BREAK : `C02D` efface les bits 0 et 2 par `AND $FA` et vide le tampon du clavier). Le bit 6 n'est pas testé par ces routines.

7 Principales routines de la ROM (p. 96-110)

Pour chaque routine, le livre donne l'entrée (registres à charger), l'action, la sortie et les registres modifiés. Dans ce chapitre, « accumulateur » désigne l'accumulateur flottant du BASIC (`044E`) et non le registre A du Z80. Le livre emploie souvent « A » pour les deux : le sens se déduit du contexte. Avant d'appeler une routine arithmétique, il faut positionner l'indicateur de type `01D9` (2 entier, 4 simple précision, 8 double précision).

7.1 Entrées/sorties

Buzzer

Adr.	Entrée	Action	Sortie	Modifiés
C2DF	IY = tonalité, H = durée, L = 0	bip (comme BEEP)	–	AF BC DE HL IY, EI

Adr.	Entrée	Action	Sortie	Modifiés
C331	IY = tonalité, HL = durée, C = FF marche / 00 arrêt	commande le haut-parleur	–	AF BC DE HL, EI

Clavier

Adr.	Entrée	Action	Sortie	Modifiés
C8C5	–	lit le clavier en affichant le curseur	A = caractère ; CY = 1 si interruption	AF
C90A	–	lit un octet au clavier (comme INKEY$)	A = caractère ; Z = 1 si aucune touche	AF

Écran LCD

Adr.	Entrée	Action	Sortie	Modifiés
C1BE	A = caractère	écrit à l'écran et dans l'image mémoire ; le curseur avance ; codes de contrôle admis	–	aucun
C231	H = X, L = Y, C = caractère	écrit C à la position indiquée ; le curseur ne bouge pas	–	AF BC DE
CE9E	–	efface l'écran (CLS)	–	AF
C18A	A = code ASCII	affiche au curseur ; le curseur avance	–	aucun
C0A1	A = 0 ou 1	0 : affiche le curseur ; 1 : le cache	–	AF BC DE

Imprimante

Adr.	Entrée	Action	Sortie	Modifiés
CFB7	–	initialise l'imprimante parallèle (mode texte, taille 2, couleur 0)	–	AF BC DE HL
CFB0	–	envoie LF + CR à l'imprimante	–	AF BC DE HL
CEF7	A = octet	envoie A à l'imprimante (teste BUSY et BREAK)	–	AF BC DE HL
CF11	A = caractère	envoie A sur le port parallèle	–	AF B E

Sous-processeur T6834

Adr.	Entrée	Action	Sortie	Modifiés
E428	A = commande	envoie une commande sans paramètre	–	AF C
C92F	A = commande, HL = paramètres, B = nombre de paramètres, C = octets de réponse, DE = tampon (26E)	envoie une commande avec paramètres	réponse en (DE)	AF BC HL ; bit 7 de A à 1 si B = 0
C0BD	–	vide les tampons du clavier et du sous-processeur	–	AF
E348	HL = adresse dans la RAM du T6834	lit un octet	A = octet lu	AF BC DE
E334	HL = adresse, A = donnée	écrit un octet dans la RAM du T6834	–	AF BC DE

Routines générales

Adr.	Entrée	Action	Sortie	Modifiés
C39D	–	initialise les ports parallèle, série, optique, imprimante et cassette	–	AF BC DE
CF2A	–	attend que BUSY du port parallèle passe à l'état bas ; contrôle OFF, BREAK et les piles	–	AF

7.2 Conversions

Adr.	Entrée	Action	Sortie	Modifiés
CAE0	accumulateur	flottant → entier (CINT)	–	tous
CB08	accumulateur	→ simple précision (CSNG)	–	tous
CB1E	entier dans l'accumulateur	entier → simple précision	–	tous
CB90	accumulateur	→ double précision (CDBL)	–	tous
F595	HL = chaîne	ASCII → entier (s'arrête au premier caractère non numérique)	DE = résultat	
BA21	HL = chaîne	ASCII → flottant ; indicateurs ajustés	accumulateur	
BB98	HL = valeur	HL → ASCII, affiché à l'écran	écran	
BE5F	entier dans l'accumulateur, HL = tampon, B et C > 6	entier → ASCII	chaîne dans le tampon	tous
BBAB	accumulateur = flottant, B = chiffres avant la virgule, C = chiffres après, A = format	flottant → ASCII	tampon (HL), en général 041E	

Format pour BBAB : A = 0 : conversion brute ; sinon bit 7 : édition ; bit 6 : 1 ; bit 5 : « * » en tête ; bit 4 : « $ » en tête ; bit 3 : signe présent ; bit 2 : signe selon la valeur ; bit 0 : notation exponentielle.

N.d.R. Vérifié dans la ROM : BBAB commence par XOR A et ignore donc A, B et C : c'est la conversion **en format libre** (FOUT), celle qu'utilisent PRINT (EA2C) et STR$ (D53D). Elle rend HL = début du texte, terminé par 00, avec une espace en tête pour un nombre positif. Le format décrit ci-dessus correspond à une autre entrée (BEEE, PRINT USING).

7.3 Arithmétique

Adr.	Entrée	Action	Sortie	Modifiés
CCC6	HL, DE ; 1D9 = 2	addition entière	HL (DE conservé) ; en cas de dépassement : accumulateur, type = 8	
CCBB	HL, DE ; 1D9 = 2	HL − DE (entier)	idem	
CCE7	HL, DE ; 1D9 = 2	multiplication entière	idem	
FA79	DE, HL	DE / HL (entier)	quotient en double précision dans l'accumulateur	
CDA9	accumulateur, BC:DE ; 1D9 = 4	addition en simple précision	accumulateur	
CDB2	accumulateur = 1er, BC:DE = 2e	accumulateur − BC:DE	accumulateur	
CDB7	idem	accumulateur × BC:DE	accumulateur	
CDC2	accumulateur = diviseur, BC:DE = dividende	BC:DE / accumulateur	accumulateur	
B20E	accumulateur, accumulateur 2 ; 1D9 = 8	addition en double précision	accumulateur	
B200	idem	accumulateur − accumulateur 2	accumulateur	

Adr.	Entrée	Action	Sortie	Modifiés
B376	idem	multiplication	accumulateur	
B42F	idem	accumulateur / accumulateur 2	accumulateur	

N.d.R. Vérifié dans la ROM : les routines « simple précision » ne calculent pas en simple précision. CDA9 range BC :DE dans l'accumulateur 2 (C en 049F, B, E, D ensuite), complète l'accumulateur par des zéros (CB98) et saute dans B20E : **le résultat est en double précision** (type 8). L'arrondi à 6 chiffres n'a lieu qu'au rangement dans une variable simple ou par CSNG. L'évaluateur choisit la routine dans une table par type (F0FF double, F10B simple, F117 entier : addition, soustraction, multiplication, division, puissance, comparaison), l'opérande gauche ayant été empilé, puis saute en F9E4. Pour la comparaison en double (CAD9), A vaut 1 si l'accumulateur est inférieur à l'accumulateur 2, −1 s'il est supérieur, 0 s'ils sont égaux. Deux comportements en découlent : un dépassement entier passe en double (PRINT 30000+30000 affiche 60000), et CINT (CAE0) **tronque** au lieu d'arrondir (CINT(2.6) = 2, CINT(-2.6) = −2).

Comparaisons

Dans ce tableau, A est le registre A du Z80.

Adr.	Entrée	Action	Sortie	Modifiés
CA7B	BC:DE = 1er, accumulateur = 2e	comparaison en simple précision	A = −1 si BC:DE > acc., 1 si <, 0 si =	
CAA5	HL, DE	comparaison entière	A = −1 si DE > HL, 1 si <, 0 si =	
CAD9	accumulateur, accumulateur 2	comparaison en double précision	A = −1 si acc. 2 > acc., 1 si <, 0 si =	
C9FB	accumulateur ; 1D9 = 8	signe en double précision	A = 1, −1 ou 0	

7.4 Fonctions mathématiques

Adr.	Entrée	Action	Sortie	Modifiés
C9DC	accumulateur	valeur absolue (ABS)	accumulateur	
CC23	accumulateur	partie entière (INT) ; 1D9 actualisé	accumulateur	

7.5 Routines système

Instructions RST

RST	Routine	Rôle
RST 00	0000	C9 (RET) : point d'arrêt de la carte moniteur XP140F
RST 08	F52F	compare l'octet (HL) à l'octet qui suit le RST : s'ils sont égaux, HL est incrémenté et le programme reprend après cet octet ; sinon, « SN ERROR »
RST 10	F537	charge (HL) dans A et incrémente HL ; CY indique un caractère numérique ou alphabétique
RST 18	C937	envoie au T6834 la commande contenue dans A (sans paramètre)
RST 20	EE45	compare HL et DE : CY = 1 si HL < DE ; CY = 0 si HL > DE ; Z = 1 si égaux
RST 28	E88F	envoie A au périphérique ouvert (en général le LCD) ; tous les registres sont conservés
RST 30	FC2F	teste le type de la donnée (voir ci-dessous)
RST 38	E906	calcul de l'adresse d'un élément de tableau

N.d.R. Vérifié dans la ROM : RST 18 saute en C9C7 et non en C937, et il ne commande pas le T6834 : il donne le signe de l'accumulateur flottant (A = 0, 1 ou FF ; Z = 1 si nul). Voir la note de la table des vecteurs (§ 6).

1D9	Type	Indicateurs	A
2	entier	NC, Z, M	−1
3	chaîne	Z, C, P	0
4	simple précision	NZ, C, P	1
8	double précision	NZ, NC, P	5

N.d.R. Vérifié dans la ROM (exécution de `FC2F`) : la ligne de l'entier est fausse. Pour un entier, RST 30 rend **C = 1, Z = 0, M** (A = −1). Ainsi, C = 1 pour l'entier, la chaîne et la simple précision, et C = 0 pour la double précision : le répartiteur des fonctions (`FBEB`) convertit en double l'argument de `SQR`, `RND`, `LOG`, `EXP`, `COS`, `SIN`, `TAN` et `ATN` par `CALL C,$CB90`. Ces routines donnent un résultat faux si on leur passe directement un entier.

Transferts de données

Adr.	Entrée	Action	Sortie	Modifiés
`CA0B`	accumulateur (simple précision)	empile l'accumulateur (1D9 non testé)	pile	
`CA1B`	BC, DE	BC:DE → accumulateur	accumulateur	
`CA26`	accumulateur	accumulateur → BC:DE	BC, DE	
`CA42`	accumulateur, HL = tampon	accumulateur → (HL)	tampon rempli	
`CA51`	B = nombre d'octets, DE = source, HL = destination	copie un bloc	HL et DE croissants	
`CA58`	idem	copie un bloc	HL et DE décroissants	
`CAEF`	HL	HL → accumulateur ; 1D9 = 2	accumulateur	
`CA5F`	accumulateur 2	accumulateur 2 → accumulateur	accumulateur	
`CA67`	accumulateur	accumulateur → accumulateur 2	accumulateur 2	

7.6 Routines secondaires

Adr.	Entrée	Action	Sortie	Modifiés
`C02D`	–	lit BREAKFLAG ; si le bit 6 est à 1 : le remet à 0, vide le tampon des touches et met Z à 1 ; sinon retour avec Z positionné	–	aucun
`C5A6`	–	contrôle des piles : bit 1 → « LOW BATTERY », bit 3 → « CARD LOW BATTERY » (Z = 0) ; sinon Z = 1	A = BREAKFLAG	aucun
`EE13`	HL = table de sauts, C = décalage	saut dans une table de vecteurs	HL modifié	AF BC DE HL
`C557`	–	lit le commutateur de la prise d'extension	HL = 4000 (RAM) ou 2000 (ROM)	AF HL
`C5C3`	–	coupe l'alimentation	–	tous
`C5CD`	A = 1 ou 2	1 : valide le programme de lancement et éteint ; 2 : l'invalide et coupe l'alimentation	–	aucun
`E327`	–	réarme la minuterie d'extinction (15 min)	–	AF B
`C62E`	HL = adresse	teste si l'adresse est en RAM	Z = 1 si oui	AF B
`C59B`	–	SLEEP : sauvegarde l'état des piles et HL (264), puis passe en SLEEP	–	aucun
`E8E8`	–	lit BREAKFLAG	Z = 1 si un bit est à 1	AF
`D5B0`	HL = chaîne (terminée par 00 ou 0D)	affiche un message (utilise la zone des chaînes)	–	
`FEF7`	HL = chaîne (terminée par 00)	affiche un message, sans la zone des chaînes	–	
`EBF2`	–	saisie au clavier avec « ? »	HL = tampon d'entrée −1	
`EBFF`	–	saisie au clavier sans « ? »	HL = tampon d'entrée −1	
`F30D`	DE = numéro de ligne	cherche une ligne BASIC	si C = Z = 1 : BC = adresse de la ligne	
`FE5E`	A = expression	évaluation entière (< 256)	DE, poids faible dans A ; « FC ERROR » si > 255	
`FFCC`	expression	évaluation entière	DE et A	

N.d.R. Le livre décrit une seconde fois `C5A6` (p. 108) avec un autre rôle (remise à zéro de BREAKFLAG et extinction) : l'une des deux adresses est probablement fausse. De même, pour `C59B`, l'adresse de sauvegarde des piles est imprimée « 226h ».

N.d.R. Vérifié dans la ROM :

- `C5A6` correspond à la *seconde* description du livre : si le bit 7 de BREAKFLAG (OFF) est à 1, il remet BREAKFLAG à 0 et coupe l'alimentation ; sinon il revient avec A = BREAKFLAG. Les messages « LOW BATTERY » (bits 1 et 3) sont affichés par `C06C` ;
- `C02D` teste le **bit 0** (BREAK) et non le bit 6 : s'il est à 1, il efface les bits 0 et 2 (`AND $FA`) et vide les tampons du clavier et du T6834 (`C0BD`) ;
- `E8E8` fait `LD A,($002B) : AND $83` : **Z = 0** (et non Z = 1) si BREAK, piles faibles ou OFF est signalé ;
- `EBF2` et `EBFF` rendent HL = `00D4` ; la ligne saisie se termine par `00` et **CY = 1 signale un appui sur BREAK** pendant la saisie (`INPUT` saute alors en `D2CA`).

Points d'entrée divers

Adresse	Rôle
`F680`	CONT général du système
`C528`	affichage du message de copyright
`C3C3`	initialisation générale (démarrage à froid)
`FB4F`	entrée d'une routine en hexadécimal
`C4D1`	« CREATE SYSTEM ? » (Y, RETURN) : efface complètement une carte mémoire
`F23D`	mode d'attente avec curseur
`F1AA`	affiche « SN ERROR »
`F1C7`	E = numéro d'erreur : affiche l'erreur correspondante

8 Périphériques (p. 111-115)

8.1 Cartes mémoire

Il existe deux types de cartes : RAM seule (4 ou 8 Ko), ou ROM (8 Ko en général) avec 4 Ko de RAM, dont 3 Ko pour l'utilisateur (1 Ko sert de zone de travail à la carte).

Broche	Signal	Broche	Signal
1	VCC1 (alimentation du X-07)	11–18	D7 . . . D0
2	VCC2 (alimentation du X-07)	19–31	A12 . . . A0
3	NC	32	masse
4	V+ (détection LOW BAT.)	33	masse
5	NC		
6	accès RAM (RAM ON)		
7	sélection RAM (active à l'état bas)		
8	$\overline{\text{WR}}$		
9	$\overline{\text{RD}}$		
10	sélection ROM (active à l'état bas)		

8.2 Imprimante graphique X-710

La X-710 possède son propre microprocesseur 4 bits, une ROM de 4096 × 10 bits et une RAM de 256 × 4 bits, avec une horloge d'environ 600 kHz. L'octet de déplacement du stylo se décompose ainsi :

Bits	Signification
7	1 : fin de déplacement ; 0 : déplacement
4–6	longueur du mouvement
3	0 : stylo levé ; 1 : stylo baissé (tracé)
0–2	direction du mouvement

Brochage du contrôleur (fig. 29, boîtier FPP 54 broches).

Broche	Nom	Fonction
1	D4	SCS (échelle) : L → échelle 1 après reset, H → échelle 0
2	D5	CRM : L → CR seul, H → CR + LF
3	D6	EXEN (validation de la ROM externe)
4–7	D7–D10	touches : avance papier, recul papier, changement de couleur, changement de stylo
8	D11	PWS (largeur du papier) : L 58 mm, H 114 mm
9	D12	CNS (nombre de couleurs) : L 1, H 4
10	D13	HLTEN (Halt Enable)
11	D14	STR (reset de la bascule Strobe)
12	D15	BSR (reset de la bascule Busy)
13–16	R40–R43	données parallèles D0–D3 (code caractère)
17–20	R50–R53	données parallèles D4–D7
21	RESET	reset
22	GND	alimentation négative
23–24	OSC	oscillateur
25	HLT	signal Halt
26	TEST	relié à VCC
27	VCC	alimentation positive
28–35	R00–R13	EXD0–EXD7 : données de la ROM externe (reliées à VCC si elle est absente)
36	INT0	entrée Strobe (front initial)
37	INT1	reliée à GND
38–41	R20–R23	RA0–RA3 : adresse de la ROM externe
42–45	R30–R33	$\overline{\text{XD}}$…$\overline{\text{XA}}$: moteur X
46–49	R60–R63	$\overline{\text{YD}}$…$\overline{\text{YA}}$: moteur Y
50	D0	$\overline{\text{PEN UP}}$
51	D1	$\overline{\text{PEN DOWN}}$
52	D2	$\overline{\text{COLOR DETECT}}$
53	D3	$\overline{\text{BAS}}$ (strobe d'adresse de la ROM externe)
54	NC	–

Le schéma complet de la X-710 (fig. 30) se trouve p. 115 du livre : contrôleur IC3, pilote de moteurs IC4, mécanisme DPG-23, alimentation AD-5 et batterie NiCd de 6 V.

8.3 Interface vidéo X-720

— 6 Ko de mémoire vidéo de `8000` à `97FF`, entièrement utilisés par les modes `SCREEN 5` et `6`. En basse résolution, deux pages d'écran sont disponibles, ce qui permet de basculer très vite de l'une à l'autre.
— 4 Ko de ROM.
— La X-720 redirige les hooks de la zone système (comme la carte XP140F) pour ajouter SCREEN, PAINT et COLOR, et pour étendre LINE, PSET, PRESET et CIRCLE.
— Le connecteur à l'arrière était prévu pour des cartouches ROM ou des extensions de RAM, mais aucune n'a été commercialisée.

N.d.R. Le livre écrit « 800h et 97FFh » pour la mémoire vidéo (il faut lire `8000`) et « 4000h à B000h » pour la ROM, alors que la carte mémoire (fig. 19) place la ROM TV en `A000`–`AFFF`.

3e partie du livre : Applications (p. 117-135)

La 3e partie du livre met en pratique les notions des deux premières : huit petites routines en langage machine Z80, chacune avec son principe, son listing assembleur, un organigramme et un programme BASIC qui charge les octets en mémoire par `POKE`. Les listings ont été vérifiés en désassemblant les octets des lignes `DATA` ; les erreurs du livre y sont corrigées.

Les routines de la ROM utilisées sont décrites à la section 7.

9 Vue d'ensemble

9.1 Les huit programmes

Fig.	Programme	Implantation	Ce qu'il fait
31	Affichage	`1B00-1B24`	dessine un motif de 8×8 points en écrivant dans la RAM LCD du T6834
32	Inversion vidéo	`1C00-1C15`	inverse la mémoire vidéo de la X-720 (SCREEN 5 et 6)
33	Inversion LCD	`1C00-1C1C`	inverse l'écran LCD
34	Affichage d'un titre	`1C00-1C2C`	fige un titre sur la première ligne de l'écran
35	Copyright personnel	`2000-2027`	affiche un message à la mise sous tension (mot-clé `love`)
36	Écriture sur la X-710	`1C00-1C26`	imprime un message sur la table traçante X-710
37	Bruitages	`1C00-1C2B`	produit des sons par le générateur de bauds
38	Redéfinition de touche	`1C00-1C20`	associe un message à une touche

Les programmes sont indépendants. Sauf le copyright, ils s'implantent tous en `1B00` ou en `1C00`, à la fin des 8 Ko de RAM interne : il faut donc les charger un par un. La zone choisie doit rester hors du programme BASIC chargeur. Si le programme BASIC est long, on peut réserver la mémoire avec `CLEAR`.

9.2 Le chargeur BASIC type

Chaque programme est fourni avec un chargeur BASIC sur le même modèle :

```
10 CLS:PRINT"un instant !"
20 FOR I=&H1C00 TO &H1C15            ' première et dernière adresse
30 READ A$:POKE I,VAL("&H"+A$)       ' un octet hexadécimal par DATA
40 NEXT I
50 EXEC &H1C00                       ' lance la routine
60 DATA ...
```

La routine se termine par `RET` et rend la main au BASIC. Dans les DATA, les octets sont donnés en hexadécimal, sans préfixe.

9.3 Routines de la ROM utilisées

Adresse	Rôle	Programmes
`CE9E`	efface l'écran (CLS)	titre, copyright
`E334`	écrit A dans la RAM du T6834, à l'adresse HL	affichage, LCD, titre
`E348`	lit dans A l'octet de la RAM du T6834 à l'adresse HL	LCD
`FEF7`	affiche la chaîne pointée par HL (terminée par `00`)	titre, copyright
`D5B0`	affiche la chaîne pointée par HL (zone des chaînes BASIC)	redéfinition
`F23D`	retour au mode « attente curseur »	copyright, redéfinition
`C1BE`	sortie d'un caractère sur la console (LCD)	redéfinition
`CFB7`	initialise l'imprimante	X-710
`CED6`	envoie A à l'imprimante (GPRPUT)	X-710

10 Affichage d'un motif 8×8 (p. 119-120)

But. Dessiner un motif de 8×8 points en écrivant directement dans la mémoire de l'écran LCD, qui appartient au T6834. On n'est plus limité aux caractères de 6×8 points. Le principe est le même que celui de l'instruction BASIC `FONT$`.

Principe. Dans la RAM de l'écran, la plus petite unité adressable est une « ligne » de 8 points horizontaux : un bit à 1 est un point allumé, un bit à 0 un point éteint. Le motif est donc décrit par une suite de 8 octets, un par ligne. La routine écrit ces octets l'un après l'autre dans la RAM du T6834 avec `E334`, en commençant à `E000` (coin haut gauche de l'écran). Elle ajoute `10` à l'adresse après chaque ligne, ce qui correspond à une ligne de points plus bas.

Implantation	`1B00`–`1B1A` (code) ; `1B1B`–`1B24` (données)
Longueur	27 octets de code + 10 octets de données
Lancement	`EXEC &H1B00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1B00	06 08		LD B,$08	nombre de lignes
1B02	21 1B 1B		LD HL,TB	adresse de la table du motif
1B05	7E	LO	LD A,(HL)	octet du motif
1B06	E5		PUSH HL	
1B07	2A 23 1B		LD HL,(AD)	adresse d'écriture dans la RAM LCD
1B0A	C5		PUSH BC	
1B0B	CD 34 E3		CALL $E334	écriture dans la RAM du T6834
1B0E	C1		POP BC	
1B0F	11 10 00		LD DE,$0010	une ligne de points plus bas
1B12	19		ADD HL,DE	
1B13	22 23 1B		LD (AD),HL	
1B16	E1		POP HL	
1B17	23		INC HL	octet suivant du motif
1B18	10 EB		DJNZ LO	
1B1A	C9		RET	
1B1B	FF 81 ...	TB	DEFB $FF,$81,$81,$81,	motif : un carré
	... 81 FF		$81,$81,$81,$FF	
1B23	00 E0	AD	DEFB $00,$E0	adresse de départ : E000

Le motif. Chaque octet code une ligne du dessin :

Dessin	Binaire	Hexa
`########`	11111111	FF
`#......#`	10000001	81
`#......#`	10000001	81
`#......#`	10000001	81
`#......#`	10000001	81
`#......#`	10000001	81
`#......#`	10000001	81
`########`	11111111	FF

Pour un autre dessin, il suffit de remplacer les 8 octets de `1B1B` à `1B22`. On change l'emplacement en modifiant les deux octets de `1B23` (adresse dans la RAM LCD, poids faible en premier).

Chargeur BASIC.

```
10 CLS:PRINT"un instant !"
20 FOR I=&H1B00 TO &H1B24
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA 06,08,21,1B,1B,7E,E5,2A,23,1B,C5,CD,34,E3,C1,11,10,00,19,22,23,1B
60 DATA E1,23,10,EB,C9,FF,81,81,81,81,81,81,FF,00,E0
```

Le chargeur ne lance pas la routine : taper `EXEC &H1B00`.

N.d.R. La routine modifie l'adresse stockée en `1B23` : pour la relancer au même endroit, il faut d'abord remettre `00,E0` dans ces deux octets (ou relancer le chargeur). Le texte renvoie au « paragraphe 7.7 », qui n'existe pas : il s'agit du § 7.6.2 (RAM du T6834).

11 Inversion de l'écran vidéo (p. 121-122)

But. Inverser les couleurs de l'écran de l'interface vidéo X-720 de façon quasi instantanée, dans les modes `SCREEN 5` et `6`.

Principe. On utilise un `XOR` avec `FF` : $1 \oplus 1 = 0$ et $0 \oplus 1 = 1$, donc chaque bit est inversé. La routine parcourt toute la mémoire vidéo, de `8000` à `97FF`, octet par octet. Cette mémoire est directement accessible au Z80, d'où la rapidité.

Implantation	`1C00`–`1C15`
Longueur	22 octets
Lancement	`EXEC &H1C00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	06 FF		LD B,$FF	masque d'inversion
1C02	21 00 80		LD HL,$8000	début de la mémoire vidéo
1C05	7E	1E	LD A,(HL)	
1C06	A8		XOR B	inversion
1C07	77		LD (HL),A	
1C08	7C		LD A,H	
1C09	FE 97		CP $97	poids fort de l'adresse de fin ?
1C0B	28 03		JR Z,FI	
1C0D	23	2E	INC HL	
1C0E	18 F5		JR 1E	
1C10	7D	FI	LD A,L	
1C11	FE FF		CP $FF	poids faible de l'adresse de fin ?
1C13	C8		RET Z	fin à `97FF`
1C14	18 F7		JR 2E	

Chargeur BASIC et démonstration. Le programme passe en `SCREEN 5`, trace un trait, puis inverse l'écran en boucle.

```
 10 CLS:PRINT"un instant !"
 20 FOR I=&H1C00 TO &H1C15
 30 READ A$:POKE I,VAL("&H"+A$)
 40 NEXT
 50 DATA 06,FF,21,00,80,7E,A8,77,7C,FE,97,28,03,23,18,F5,7D,FE,FF,C8,18,F7
 60 SCREEN 5
 70 PRINT"INVERSE VIDEO"
 80 FOR I=0 TO 100
 90 PSET (I,50):NEXT
100 EXEC&H1C00
110 FOR I=0 TO 500:NEXT:GOTO 100
```

Ce programme nécessite l'interface X-720.

12 Inversion de l'écran LCD (p. 123-124)

But. Même chose pour l'écran LCD : inverser chaque octet de sa mémoire par un `XOR` avec `FF`.

Principe. La mémoire de l'écran LCD n'est pas adressée par le NSC800 mais par le T6834. On passe donc par les routines `E348` (lecture) et `E334` (écriture) de la ROM, octet par octet, de `E000` à `E1FE`. Les octets `E00F`, `E01F` et `E02F` n'ont pas besoin d'être inversés. Les auteurs les inversent quand même : les tester prendrait plus de temps que de les traiter.

Implantation	`1C00`–`1C1C`
Longueur	29 octets
Lancement	`EXEC &H1C00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	21 FF DF		LD HL,$DFFF	début de la RAM LCD −1
1C03	23	1E	INC HL	
1C04	CD 48 E3		CALL $E348	lecture dans la RAM LCD
1C07	06 FF		LD B,$FF	
1C09	A8		XOR B	inversion
1C0A	CD 34 E3		CALL $E334	écriture dans la RAM LCD
1C0D	E5		PUSH HL	
1C0E	7C		LD A,H	
1C0F	FE E1		CP $E1	poids fort de l'adresse de fin ?
1C11	28 03		JR Z,FI	
1C13	E1		POP HL	
1C14	18 ED		JR 1E	
1C16	7D	FI	LD A,L	
1C17	FE FE		CP $FE	poids faible de l'adresse de fin ?
1C19	E1		POP HL	
1C1A	C8		RET Z	fin à E1FE
1C1B	18 E6		JR 1E	

Chargeur BASIC.

```
10 CLS:PRINT"un instant"
20 FOR I=&H1C00 TO &H1C1C
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT
45 EXEC&H1C00
50 DATA 21,FF,DF,23,CD,48,E3,06,FF,A8,CD,34,E3,E5,7C,FE,E1,28,03,E1,18,ED,7D
60 DATA FE,FE,E1,C8,18,E6
```

N.d.R. Le livre écrit `FOR I=&H1C00 TO &H1C1B` : le dernier octet (`E6`, à `1C1C`) n'est alors jamais écrit, et l'instruction `JR` finale saute n'importe où. La ligne 20 est corrigée ici.

Attention. Sur certaines versions du X-07, des octets échappent à l'inversion quand le T6834 est débordé par le flux de données. Dans ce cas, on peut utiliser à la place la commande `13` du T6834 (PEOR, qui inverse un point).

13 Affichage d'un titre (p. 125-126)

But. Figer un message sur la première ligne de l'écran. Ce « titre » ne disparaît pas quand l'écran défile ; seul un `CLS` l'efface.

Principe. Après un effacement de l'écran (`CE9E`), la routine écrit dans la RAM du T6834 les limites de la zone de défilement : ligne de début 1 en `C09E`, ligne de fin 3 en `C096`. La ligne 0 n'est donc plus concernée par le défilement. Le message est ensuite affiché par `FEF7`. Il doit se terminer par `00`.

Implantation	`1C00`–`1C19` (code) ; `1C1A`–`1C2C` (message)
Longueur	26 octets de code + 19 octets de message
Lancement	lancé par le chargeur

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	CD 9E CE		CALL $CE9E	efface l'écran
1C03	3E 01		LD A,$01	début du défilement : ligne 1
1C05	21 9E C0		LD HL,$C09E	adresse dans la RAM du T6834
1C08	CD 34 E3		CALL $E334	
1C0B	3E 03		LD A,$03	fin du défilement : ligne 3
1C0D	21 96 C0		LD HL,$C096	
1C10	CD 34 E3		CALL $E334	
1C13	21 1A 1C		LD HL,ME	
1C16	CD F7 FE		CALL $FEF7	affichage du titre
1C19	C9		RET	
1C1A	20 2A ...	ME	DEFM " *** PROGRAMME ***"	message
1C2C	00		DEFB $00	fin du message

Chargeur BASIC.

```
10 CLS:PRINT"un instant !"
20 FOR I=&H1C00 TO &H1C2C
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT I:EXEC&H1C00
50 DATA CD,9E,CE,3E,01,21,9E,C0,CD,34,E3,3E,03,21,96,C0,CD,34,E3,21,1A,1C
60 DATA CD,F7,FE,C9,20,2A,2A,2A,20,50,52,4F,47,52,41,4D,4D,45,20,2A,2A,2A,00
```

Pour un autre titre, remplacer les codes ASCII de la ligne 60 (20 caractères au plus, terminés par 00) et ajuster la borne de la ligne 20.

N.d.R. Le livre annonce 44 octets, de 1C00 à 1C2B. Avec le 00 final, le programme occupe en réalité 45 octets, jusqu'à 1C2C, comme l'indique la ligne 20.

14 Copyright personnel (p. 127-128)

But. Afficher un message personnel à chaque mise sous tension du X-07.

Principe. À l'initialisation, la ROM cherche le mot-clé love tous les 800 octets à partir de 2000 (voir § 7.4 du livre). Les 6 octets qui suivent le mot-clé sont trois adresses : les deux premières sont appelées au démarrage, la troisième après un SLEEP. Ici, la première adresse pointe vers la routine d'affichage. Les deux autres pointent vers un simple saut au mode « attente curseur ».

Emplacement. La routine doit commencer à une adresse testée par la ROM : 2000, 2800, 3000, 3800, 4000, 4800 ou 5000. Il faut donc de la RAM à cet endroit (carte RAM ou extension de 8 Ko). La routine est relogeable, mais les trois adresses qui suivent love doivent être modifiées en conséquence. Par exemple, à partir de 3000 :

```
ORG  $3000
DEFM love
DEFB $0A,$30
DEFB $19,$30
DEFB $19,$30
```

Implantation	2000–2027 (en-tête 2000–2009, code 200A–201B, message 201C–2027)
Longueur	40 octets
Lancement	mettre le X-07 hors tension puis sous tension

Adr.	Octets	Étiq.	Instruction	Commentaire
2000	6C 6F 76 65		DEFM love	mot-clé cherché par la ROM
2004	0A 20		DEFB $0A,$20	1er sous-programme : 200A
2006	19 20		DEFB $19,$20	2e sous-programme : 2019

Adr.	Octets	Étiq.	Instruction	Commentaire
2008	19 20		DEFB $19,$20	sous-programme après SLEEP : 2019
200A	CD 9E CE		CALL $CE9E	efface l'écran
200D	21 02 03		LD HL,$0302	X = 2, Y = 3
2010	22 B8 00		LD ($00B8),HL	position du curseur
2013	21 1C 20		LD HL,ME	
2016	CD F7 FE		CALL $FEF7	affichage du message
2019	C3 3D F2		JP $F23D	attente curseur
201C	76 69 ...	ME	DEFM "vive le X07"	message
2027	00		DEFB $00	fin du message

Chargeur BASIC. Le programme éteint le X-07 (OFF) : le message s'affiche au rallumage.

```
10 CLS:PRINT"un instant"
20 FOR I=&H2000 TO &H2027
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT I
50 OFF
60 DATA 6C,6F,76,65,0A,20,19,20,19,20,CD,9E,CE,21,02,03,22,B8,00,21,1C,20
70 DATA CD,F7,FE,C3,3D,F2,76,69,76,65,20,6C,65,20,58,30,37,00
```

N.d.R. Le livre annonce une routine de 29 octets, de 200A à 2026. Elle va en réalité jusqu'à 2027 (30 octets, une correction manuscrite du lecteur sur le scan l'indique aussi), soit 40 octets avec l'en-tête.

15 Écriture sur la table traçante X-710 (p. 129-130)

But. Imprimer un message sur la X-710. L'intérêt est surtout de montrer comment utiliser les routines de l'imprimante.

Principe. L'imprimante est initialisée par CFB7. Les caractères du message sont ensuite envoyés un par un à CED6, jusqu'au code 00 qui marque la fin. Les registres sont sauvegardés autour de chaque appel.

Implantation	1C00–1C17 (code) ; 1C18–1C26 (message)
Longueur	24 octets de code + 15 octets de message
Lancement	EXEC &H1C00

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	CD B7 CF		CALL $CFB7	initialisation de l'imprimante
1C03	21 18 1C		LD HL,TB	
1C06	7E	1E	LD A,(HL)	
1C07	FE 00		CP $00	fin de la chaîne ?
1C09	C8		RET Z	
1C0A	F5		PUSH AF	
1C0B	C5		PUSH BC	
1C0C	D5		PUSH DE	
1C0D	E5		PUSH HL	
1C0E	CD D6 CE		CALL $CED6	envoi à l'imprimante
1C11	E1		POP HL	
1C12	D1		POP DE	
1C13	C1		POP BC	
1C14	F1		POP AF	
1C15	23		INC HL	
1C16	18 EE		JR 1E	
1C18	42 4F ...	TB	DEFM "BONJOUR DE X07"	message
1C26	00		DEFB $00	fin du message

Chargeur BASIC.

```
10 CLS:PRINT"un instant !"
20 FOR I=&H1C00 TO &H1C26
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT I
60 DATA CD,B7,CF,21,18,1C,7E,FE,00,C8,F5,C5,D5,E5,CD,D6,CE,E1,D1,C1,F1,23
70 DATA 18,EE,42,4F,4E,4A,4F,55,52,20,44,45,20,58,30,37,00
```

Le chargeur ne lance pas la routine : taper `EXEC &H1C00`, la X-710 étant branchée et allumée.

16 Bruitages (p. 131-132)

But. Produire avec le buzzer des sons impossibles à obtenir avec `BEEP`, en programmant directement les ports d'E/S. Le livre rappelle un article de la gazette n° 1 du Club C7 qui produisait « des sons assez inattendus ».

Principe. Le programme utilise trois ports du circuit d'E/S (voir § 7.5 du livre et la section 3) :

- **port `F4`** : on y écrit `4E`, soit BRGST = 1 (le générateur de bauds compte), MD1 = MD0 = 1 (sortie sur le buzzer) et BZON = 1 (buzzer en marche) ;
- **ports `F2` et `F3`** : ils fixent la fréquence des impulsions. C'est un compteur de 12 bits : 8 bits dans `F2`, 4 bits dans `F3`.

La routine fait varier `F2` de `FF` à 1 pour chaque valeur de `F3` (d'abord 0, puis de `0E` à 1). Une petite temporisation (TEMPO) rend les sons audibles. Pour arrêter le buzzer, on écrit `00` dans `F4`.

Implantation	`1C00`–`1C2B`
Longueur	44 octets
Lancement	`EXEC &H1C00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	3E 00		LD A,$00	
1C02	06 0F		LD B,$0F	valeur de départ de F3
1C04	D3 F3		OUT ($F3),A	
1C06	3E 4E		LD A,$4E	BRGST, MD1, MD0, BZON
1C08	D3 F4		OUT ($F4),A	buzzer en marche
1C0A	3E FF	2E	LD A,$FF	
1C0C	D3 F2	1E	OUT ($F2),A	poids faible de la fréquence
1C0E	3D		DEC A	
1C0F	28 05		JR Z,F3	
1C11	CD 24 1C		CALL TE	temporisation
1C14	18 F6		JR 1E	
1C16	78	F3	LD A,B	
1C17	3D		DEC A	
1C18	28 05		JR Z,FI	
1C1A	D3 F3		OUT ($F3),A	poids fort de la fréquence
1C1C	47		LD B,A	
1C1D	18 EB		JR 2E	
Arrêt				
1C1F	3E 00	FI	LD A,$00	
1C21	D3 F4		OUT ($F4),A	buzzer arrêté
1C23	C9		RET	
Temporisation				
1C24	F5	TE	PUSH AF	
1C25	3E 80		LD A,$80	
1C27	3D	LO	DEC A	
1C28	20 FD		JR NZ,LO	
1C2A	F1		POP AF	
1C2B	C9		RET	

Chargeur BASIC.

```
10 PRINT"un instant !"
20 FOR I=&H1C00 TO &H1C2B
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT
50 EXEC&H1C00
60 DATA 3E,00,06,0F,D3,F3,3E,4E,D3,F4,3E,FF,D3,F2,3D,28,05,CD,24,1C
70 DATA 18,F6,78,3D,28,05,D3,F3,47,18,EB,3E,00,D3,F4,C9,F5,3E,80,3D,20,FD,F1,C9
```

On obtient d'autres sons en changeant les valeurs de départ (`1C01`, `1C03`, `1C0B`) ou la durée de la temporisation (`1C26`).

N.d.R. Le livre parle des « ports de sortie du NSC800 » : ce sont en fait les ports du circuit d'E/S HD61L202F. Il écrit aussi « 8 octets dans le port F2 et 4 autres dans le port F3 » : il s'agit de bits.

17 Redéfinition d'une touche (p. 133-134)

But. Associer un message à une touche du clavier. Ici, la touche ~ (tilde) affiche « X07 ». La même méthode permet de redéfinir tout le clavier, par exemple en AZERTY.

Principe. Tout caractère affiché passe par la sortie console, appelée par le saut placé en `009F` dans la zone système (`JP C1BE`, voir § 8.3 du livre). La routine se compose de deux parties :

1. l'installation remplace l'adresse de ce saut (octets `00A0` et `00A1`) par celle de la routine personnelle ;
2. la routine personnelle compare le caractère reçu dans `A` au code de la touche redéfinie (`7E`). S'ils sont égaux, elle affiche le message par `D5B0` et revient au mode « attente curseur » ; sinon, elle passe la main à la routine d'origine `C1BE`, qui affiche le caractère normalement.

L'organigramme n'a pas de fin : une fois installée, la routine est appelée à chaque caractère affiché et décide elle-même si elle doit agir.

Implantation	`1C00`–`1C1B` (code) ; `1C1C`–`1C20` (message)
Longueur	28 octets de code + 5 octets de message
Lancement	lancé par le chargeur

Adr.	Octets	Étiq.	Instruction	Commentaire
Changement du saut (hook)				
1C00	21 07 1C		LD HL,DB	adresse de la routine personnelle
1C03	22 A0 00		LD ($00A0),HL	remplace l'adresse du JP en 009F
1C06	C9		RET	
Routine personnelle				
1C07	F5	DB	PUSH AF	
1C08	FE 7E		CP "~"	touche redéfinie ?
1C0A	28 04		JR Z,AF	
1C0C	F1		POP AF	
1C0D	C3 BE C1		JP $C1BE	affichage normal du caractère
1C10	E5	AF	PUSH HL	
1C11	21 1C 1C		LD HL,ME	
1C14	CD B0 D5		CALL $D5B0	affichage du message
1C17	E1		POP HL	
1C18	F1		POP AF	
1C19	C3 3D F2		JP $F23D	attente curseur
1C1C	20 58 30 37	ME	DEFM " X07"	message
1C20	00		DEFB $00	fin du message

Chargeur BASIC.

```
10 PRINT"un instant"
20 FOR I=&H1C00 TO &H1C20
```

```
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT
50 EXEC&H1C00:PRINT"APPUYEZ SUR ~"
60 DATA 21,07,1C,22,A0,00,C9,F5,FE,7E,28,04,F1,C3,BE,C1,E5,21,1C,1C,CD,B0,D5
70 DATA E1,F1,C3,3D,F2,20,58,30,37,00
```

Remarques.

— Comme la routine intercepte la sortie console, le message apparaît aussi quand un programme affiche le caractère ~, par exemple `PRINT "~"`. C'est d'ailleurs ce que fait la ligne 50 du chargeur.
— Pour revenir au fonctionnement normal, il faut remettre l'adresse d'origine : `POKE &HA0,&HBE:POKE &HA1,&HC1`.

N.d.R. Le texte indique que la routine d'origine est en `CEB1`, mais le code saute en `C1BE`, l'adresse du saut d'origine en `009F`. Le livre annonce aussi 32 octets (`1C00`–`1C1F`) : le programme occupe en réalité 33 octets, jusqu'à `1C20`.

18 Conclusion du livre (p. 135)

La 3e partie se termine sur un court récit daté d'octobre 1985 : CANDORE, qui s'était endormi devant son X-07, se réveille et voit l'écran clignoter, le buzzer émettre des sons inconnus du BASIC, l'imprimante tracer à grande vitesse et le téléviseur afficher « EUREKA ». Grâce au « vieux sage ASSEMBLUS » et à l'ouvrage *le Canon X-07 et l'assembleur*, il a découvert tous les secrets de sa machine. Le texte est signé A. Tonic.

Viennent ensuite les annexes, qui ne sont pas reprises ici : liste des codes Z80 (annexe 1), assembleurs et désassembleurs disponibles (annexe 2), présentation du Club C7 (annexe 3) et bibliographie (annexe 4).